

DEVOPS
CI/CD
AUTOMATION
INFRASTRUCTURE
PRODUCTION



LEARN
PRACTICE
BUILD
DEPLOY
GROW

JENKINS PIPELINES IN PRODUCTION

A COMPLETE 20-PAGE JUNIOR ENGINEER NOTEBOOK



Code



Build



Test



Package



Deploy



REAL-WORLD
EXAMPLES



PRODUCTION
SCENARIOS



SECURITY
BEST PRACTICES



PRACTICAL
EXERCISES



TROUBLESHOOTING
GUIDE



JUNIOR ENGINEER
FRIENDLY

FROM ZERO TO PRODUCTION



Git



Docker



Kubernetes



SonarQube



Artifactory



AWS / Azure / GCP

- ☒ BUILD SKILLS
- ☒ SOLVE PROBLEMS
- ☒ WORK IN PRODUCTION
- ☒ BECOME A BETTER ENGINEER

SAME
ENGINEERING
MINDSET
DIFFERENT RESULTS

JENKINS PIPELINES IN PRODUCTION

AUTOMATE
TEST
DEPLOY
DELIVER
REPEAT

UNDERSTAND • BUILD • DEPLOY • OPERATE

WHAT IS JENKINS?

- Jenkins is an open-source automation server used to build, test and deploy applications.
- It helps teams automate the software delivery process using pipelines.
- In production, Jenkins coordinates multiple tools to deliver code reliably, safely and repeatedly.

WHY TEAMS USE JENKINS

- Automates repetitive tasks.
- Works with almost any tool or technology.
- Supports complex, real-world pipelines.
- Helps deliver software faster and more reliably.
- Widely used in production across industries.

CI/CD EXPLAINED

- **CI (Continuous Integration):** Developers regularly merge code into a shared repository. Jenkins automatically builds and tests the code.
- **CD (Continuous Delivery/Deployment):** Jenkins helps package, release and deploy the application to different environments (dev, staging, production).

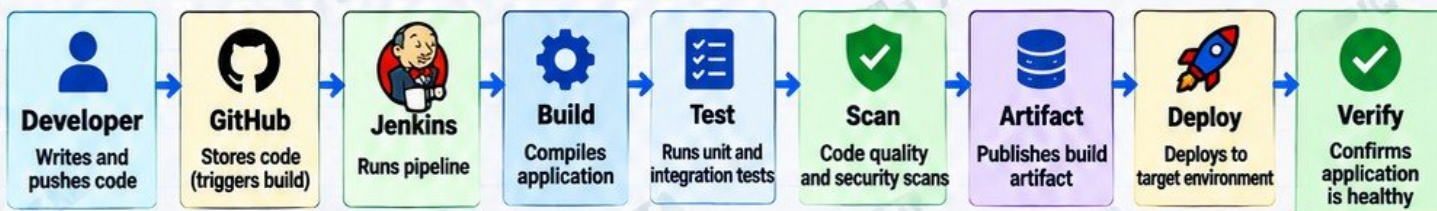
PRODUCTION PIPELINES ARE MORE THAN SHELL COMMANDS

- Real pipelines include multiple tools, environments, security checks, approvals and error handling.
- They must be reliable, secure, maintainable and scalable.
- Production pipelines handle failures, rollbacks, notifications and audits.
- They follow best practices, not just a list of shell commands.

JENKINS COMPONENTS

	Controller	Manages jobs, schedules builds, and coordinates agents.
	Agents	Execute pipeline jobs (build, test, deploy). Can be physical, virtual, Docker or Kubernetes.
	Repositories	Source code is stored in Git (GitHub, GitLab, Bitbucket, etc).
	Artifact Systems	Store build artifacts (e.g. JFrog Artifactory, Nexus, Docker Registry).
	Deployment Targets	Deploy to servers, Kubernetes, cloud platforms (AWS, Azure, GCP) or other environments.

BASIC PRODUCTION PIPELINE FLOW



DEVOPS CONNECTION

- Jenkins coordinates tools in the DevOps ecosystem.
- It works with Git, build tools, test frameworks, security scanners, artifact repositories, deployment tools and monitoring systems.
- Jenkins does not replace these tools. It brings them together.
- It helps teams deliver value to users faster and more reliably.

KEY TAKEAWAY



JENKINS IS THE ORCHESTRATOR.
IT CONNECTS PEOPLE, CODE, TOOLS
AND ENVIRONMENTS TO DELIVER
SOFTWARE IN PRODUCTION.
IT DOES NOT REPLACE THEM.

HOW JENKINS PIPELINE ARCHITECTURE WORKS

COMPONENTS • FLOW • ROLES • REAL-WORLD CONTEXT

KEY COMPONENTS



JENKINS CONTROLLER

The central server that manages jobs, schedules builds, stores configuration, plugins and coordinates agents.



JENKINS AGENTS

Machines that execute pipeline jobs (build, test, deploy). Agents can be physical, virtual, Docker or Kubernetes.



EXECUTORS

The number of concurrent builds an agent can run. Each executor runs one job at a time.



WORKSPACES

A directory on the agent where the code is checked out and the build runs. Each job has its own workspace.



JOBS

A task in Jenkins that defines what to run (e.g. freestyle job, pipeline job, multibranch job).



PIPELINES

Defined as code (Jenkinsfile) with stages to build, test, scan, deploy and more. Supports declarative and scripted pipelines.



PLUGINS

Extend Jenkins functionality (Git, Docker, Kubernetes, Slack, SonarQube, etc).



CREDENTIALS

Securely store secrets such as SSH keys, tokens, usernames and passwords. Used in pipelines without exposing sensitive data.



BUILD QUEUE

Holds builds waiting for an available agent and executor. Helps manage load when many jobs are triggered.

WHY SEPARATE ORCHESTRATION FROM EXECUTION?

- The controller focuses on scheduling, coordination and management.
- Agents handle the heavy work (builds, tests, deployments).
- This improves stability, performance and scalability.
- If an agent fails, the controller can still manage other agents.
- It allows the use of different environments (Linux, Windows, Docker, Kubernetes).

JENKINS PIPELINE FLOW



Git Repository

(GitHub, GitLab, Bitbucket)
Stores source code

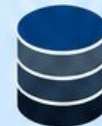
↓ Webhook



Jenkins Controller

Receives webhook, schedules builds and coordinates agents

↓



Build Queue

Holds jobs waiting for an available agent

↓



Jenkins Agent

Executes the pipeline job (build, test, deploy)

↓



Pipeline Stages

Build → Test → Scan →
Package → Deploy

↓



Deployment Target

Servers, Kubernetes, AWS,
Azure, GCP or other environments

PRODUCTION AWARENESS



AVOID RUNNING HEAVY BUILDS DIRECTLY ON THE CONTROLLER.

- Can make Jenkins slow or unstable.
- May lead to disk space issues.
- Use dedicated agents for builds and tests.
- Keep the controller for orchestration only.

YOUR FIRST JENKINSFILE

PIPELINE AS CODE FOR REAL DELIVERY

WHAT IS A JENKINSFILE?

- A Jenkinsfile is a text file that defines your CI/CD pipeline using code.
- It tells Jenkins what to do: build, test, scan, deploy and more.
- It lives in your source code repository (e.g. GitHub) with your application code.

PIPELINE AS CODE

- Infrastructure and delivery logic are defined in code, not in the Jenkins UI.
- Makes pipelines version-controlled, reviewable and repeatable.
- Allows teams to track changes, rollback and collaborate like application code.

DECLARATIVE vs SCRIPTED PIPELINE

DECLARATIVE (Recommended)

- Simpler syntax
- Easier for beginners
- Built-in structure and rules
- Ideal for most use cases

SCRIPTED

- More flexible
- Uses Groovy syntax
- Better for complex logic
- Steeper learning curve

WHY VERSION PIPELINES WITH CODE?

- Keeps application code and pipeline logic together.
- Enables code reviews and team collaboration.
- Tracks changes and provides history.
- Makes it easy to reproduce and rollback.
- Supports the same branching and release strategy as your application.

PRACTICE: CREATE A JENKINSFILE

- 1 In your project repository, create a file named Jenkinsfile
- 2 Add the pipeline code (start with the basic example)
- 3 Commit the file to Git
- 4 Push to GitHub
- 5 Configure a Jenkins job to use this repository
- 6 Run the pipeline and check the build output

BASIC JENKINSFILE EXAMPLE

```
pipeline {
    agent any

    stages {
        stage('Build') {
            steps {
                sh './build.sh'
            }
        }
    }
}
```

EXPLANATION OF KEY SECTIONS

pipeline

Defines the start of a declarative pipeline. It tells Jenkins this is a pipeline script.

agent

Specifies where the pipeline will run. agent any means any available agent can execute the pipeline.

stages

Groups all the stages in the pipeline. A pipeline can have multiple stages (e.g. Build, Test, Deploy).

stage

Defines a single stage in the pipeline. In the example, stage('Build') runs build-related tasks.

steps

Contains the actual commands to run in that stage. In the example, sh './build.sh' executes a shell script.

JUNIOR ENGINEER TIP

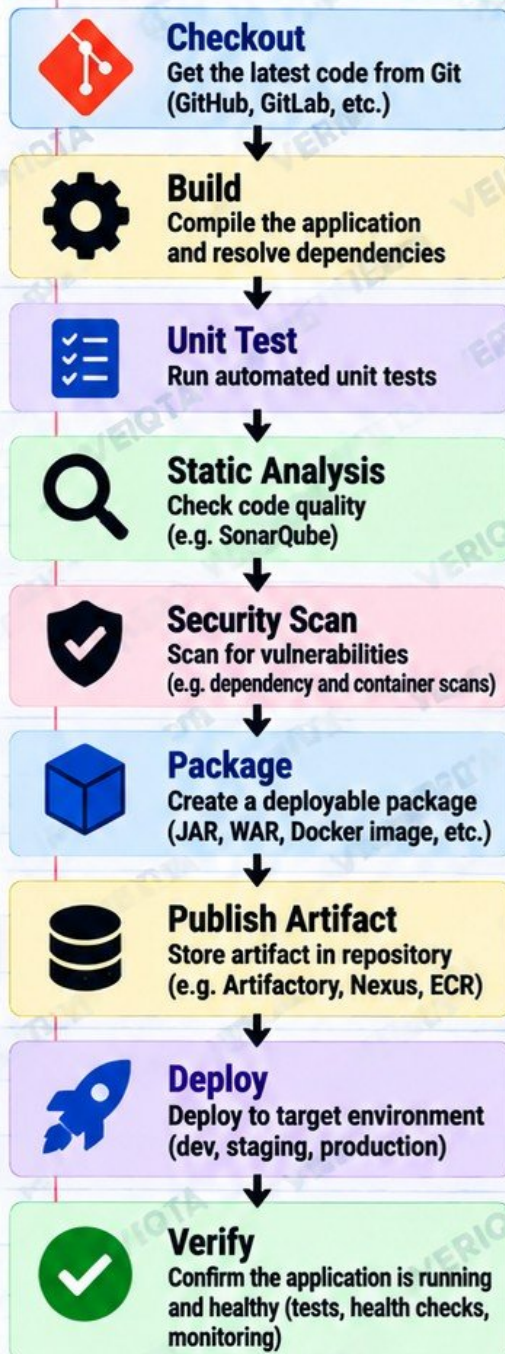


- Start with a simple pipeline and add more stages gradually.
- Keep your Jenkinsfile in the root of your repository.
- Use meaningful stage names (Build, Test, Deploy).
- Test your pipeline in a non-production environment first.

DESIGNING PRODUCTION PIPELINE STAGES

SMALL STEPS • CLEAR STAGES • RELIABLE DELIVERY

THE DELIVERY LIFECYCLE



WHAT BELONGS IN EACH STAGE?

STAGE	KEY ACTIVITIES
Checkout	Clone repository, checkout correct branch/commit
Build	Install dependencies, compile source code
Unit Test	Run unit tests, generate test reports
Static Analysis	Run code quality analysis (e.g. SonarQube)
Security Scan	Scan for vulnerabilities (e.g. SCA, container scan)
Package	Create build artifacts (JAR, WAR, Docker image)
Publish Artifact	Upload artifact to repository (e.g. Artifactory)
Deploy	Deploy to target environment (Kubernetes, AWS, etc.)
Verify	Run smoke tests, health checks, verify application

FAIL-FAST DESIGN

- Stop the pipeline early when a stage fails.
- Prevents wasting time and resources.
- Avoids deploying broken or insecure code.
- Makes issues easier and faster to fix.

WHY CLEAR STAGES?

- Each stage has a specific purpose.
- Easier to understand and maintain.
- Simplifies troubleshooting.
- Allows teams to reuse, skip or run stages independently.
- Improves visibility and reporting.
- Supports scaling and parallel execution.

BEGINNER MISTAKE

- ✗ Creating one enormous Build stage containing everything.
- Hard to maintain.
- Difficult to troubleshoot.
- Slow feedback when something fails.
- No clear separation of responsibilities.
- Not suitable for production environments.

BEST PRACTICE

- ✓ Break the pipeline into clear stages.
- ✓ Keep stages small and focused.
- ✓ Use meaningful stage names.
- ✓ Fail early and fix problems quickly.
- ✓ Follow the same structure across projects.
- ✓ Make the pipeline readable for other engineers.

"A WELL-DESIGNED PIPELINE IS EASIER TO MAINTAIN, MORE RELIABLE AND SAFER FOR PRODUCTION."

JENKINS AGENTS AND BUILD EXECUTION

DISTRIBUTE WORK • SCALE BUILDS • IMPROVE RELIABILITY

WHAT IS AN AGENT?



- An agent is a machine that executes Jenkins pipeline jobs (build, test, deploy).
- Agents can be physical, virtual, containers or Kubernetes pods.
- The controller does not run the build steps. The agent executes them.

CONTROLLER VS AGENT RESPONSIBILITIES



Controller (Master)

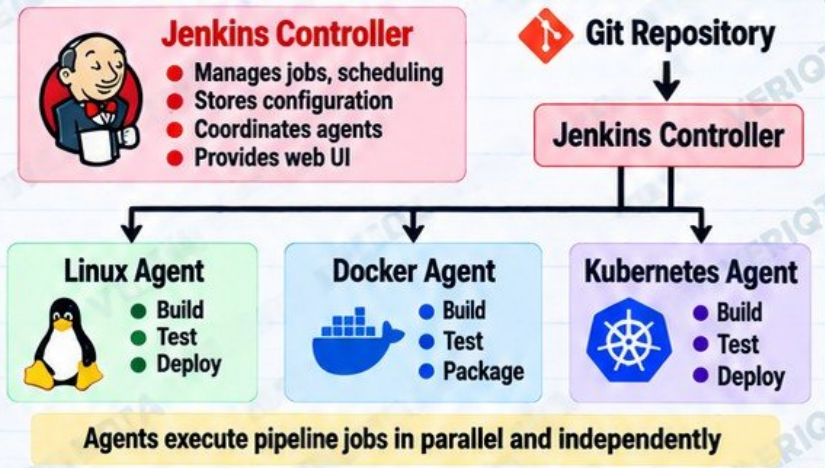
- Manages jobs and configuration
- Schedules builds
- Manages credentials and plugins
- Provides web interface
- Should not run heavy builds



Agent (Worker)

- Executes the actual build steps
- Runs tools (Java, Maven, Docker, etc.)
- Can be physical, virtual, Docker or Kubernetes
- Can be dynamically created
- Can run in parallel with other agents

JENKINS CONTROLLER AND AGENTS



STATIC AGENTS



- Always available
- Manually configured
- Good for stable workloads
- Runs on fixed machines
- Needs maintenance

EPHEMERAL AGENTS



- Created when needed
- Automatically removed after the job
- Save resources
- Common in Docker and Kubernetes
- Ideal for scalable builds

LABELS



- Labels help Jenkins find the right agent
- You can assign labels (e.g. linux, docker, k8s)
- Use labels in your Jenkinsfile
- Allows different jobs to run on specific agents

EXECUTORS



- Each agent has a number of executors.
- An executor runs one job at a time.
- If all executors are busy, new jobs wait in the queue.
- Increase executors or add more agents to handle more builds.

WORKSPACE ISOLATION



- Each job runs in its own workspace directory on the agent.
- Keeps builds isolated from each other.
- Prevents file conflicts.
- Workspaces are usually cleaned after the job (can be configured).

TYPES OF AGENTS



- Linux agents – most common, flexible and lightweight.
- Docker-based agents – run in Docker containers with required tools.
- Kubernetes-based agents – dynamically created pods, scalable and widely used in production.

SELECTING AN AGENT IN A JENKINSFILE

```

pipeline {
  agent { label 'linux' } // use a specific label
  stages {
    stage('Build') {
      steps {
        sh './build.sh'
      }
    }
  }
}
  
```

- Use agent any to run on any available agent.
- Use a label to target a specific agent (e.g. linux, docker, k8s).
- You can also define agents at the stage level.
- Choose the right agent based on the tools your build needs.

COMMON FAILURE



Pipeline remains stuck with "Waiting for next available executor."

- All executors are busy.
- Not enough agents.
- Agents are offline.
- Jobs are taking too long.
- Executor count is too low.
- Misconfigured labels.
- Stuck or hung builds.

HOW TO FIX



- Check agent status (Manage Nodes).
- Increase the number of executors.
- Add more agents (static or dynamic).
- Use labels correctly.
- Terminate stuck builds.
- Use ephemeral agents for scaling.
- Monitor queue length and build duration.
- Consider using Kubernetes agents for auto-scaling.

PARAMETERS, VARIABLES AND BUILD CONFIGURATION



MAKE YOUR PIPELINES FLEXIBLE • REUSABLE • PRODUCTION-READY

ENVIRONMENT VARIABLES



- Environment variables store key-value pairs used during the pipeline run.
- They can be defined in the Jenkinsfile, in Jenkins or injected from credentials.
- Used for configuration, tool paths, feature flags and deployment settings.

```
pipeline {  
  agent any  
  environment {  
    APP_ENV = 'dev'  
    APP_NAME = 'myapp'  
  }  
}
```

JENKINS BUILT-IN VARIABLES



- Jenkins provides many built-in variables that you can use in your pipeline.
- They give you information about the current build, branch, commit, user and more.

VARIABLE	DESCRIPTION
BUILD_NUMBER	The build number
BUILD_ID	Unique build ID
JOB_NAME	Name of the job
BRANCH_NAME	Git branch name
GIT_COMMIT	Commit SHA
GIT_URL	Repository URL
BUILD_URL	Link to the build
NODE_NAME	Agent name
WORKSPACE	Pipeline workspace

PARAMETERS {}



- Parameters allow you to provide input when triggering a build.
- They make pipelines flexible and reusable across environments.

```
pipeline {  
  agent any  
  parameters {  
    string(name: 'ENV', defaultValue: 'dev',  
      description: 'Target environment')  
    booleanParam(name: 'RUN_TESTS',  
      defaultValue: true,  
      description: 'Run unit tests')  
    choice(name: 'APP_VERSION',  
      choices: ['1.0', '1.1', '2.0'],  
      description: 'Select version')  
  }  
  stages { ... }  
}
```

T STRING PARAMETERS

- Used for text input such as environment name, version or tag.

```
string(name: 'ENV',  
  defaultValue: 'dev',  
  description: 'Target environment')
```

BOOLEAN PARAMETERS

- Used for true/false values.
- Common for optional steps (e.g. run tests, deploy, notify).

```
booleanParam(name: 'RUN_TESTS',  
  defaultValue: false,  
  description: 'Run unit tests')
```

CHOICE PARAMETERS

- Provides a list of predefined options.
- Useful for selecting environment, region or version.

```
choice(name: 'ENV',  
  choices: ['dev', 'staging', 'prod'],  
  description: 'Select environment')
```

BUILD NUMBER

- Automatically incremented for every build.
- Useful for versioning and tracking.

```
echo "Build number: ${BUILD_NUMBER}"
```

BRANCH NAME

- Shows the Git branch being built.
- Useful in multibranch pipelines.

```
echo "Branch: ${BRANCH_NAME}"
```

COMMIT INFORMATION

- Provides the current commit SHA.
- Useful for tagging artifacts and deployments.

```
echo "Commit: ${GIT_COMMIT}"
```

WHY NOT HARDCODE CONFIGURATION?



- Hardcoding values (e.g. environment, version, URLs) makes pipelines difficult to maintain.
- Use parameters and environment variables instead.
- It allows the same pipeline to work across dev, staging and production.
- Improves flexibility, security and reusability.

SECURITY REMINDER



- Environment variables are **NOT** automatically safe places for secrets.
- Environment variables can be seen in build logs if not handled correctly.
- Use Jenkins **Credentials** for sensitive data (e.g. passwords, tokens, keys).
- Follow least privilege and avoid printing secrets in the logs.

PRACTICE

- 1 Add parameters to your Jenkinsfile.
- 2 Use built-in variables in a stage.
- 3 Print the build number, branch name and commit SHA.
- 4 Run a build with different parameter values.
- 5 Observe the output in the console log.

GIT, GITHUB AND JENKINS INTEGRATION

CONNECT CODE TO AUTOMATION • BUILD FASTER • DELIVER RELIABLY

KEY CONCEPTS



REPOSITORY CHECKOUT

- Jenkins checks out your source code from Git/GitHub.
- Uses the correct branch, commit or PR.



BRANCH DISCOVERY

- Jenkins can automatically discover branches in a repository.
- Important for Multibranch Pipelines.



WEBHOOKS

- GitHub sends an event to Jenkins when code is pushed, a PR is created or updated.
- Triggers a build automatically.



MULTIBRANCH PIPELINES

- Automatically creates a pipeline for each branch.
- Detects new branches and PRs.



PULL REQUESTS

- Jenkins can build and test pull requests.
- Helps catch issues before merging.



COMMIT-TRIGGERED BUILDS

- Each push to a branch can trigger a build.
- Ensures code is tested early.



BRANCH-BASED BEHAVIOR

- Run different stages for different branches (e.g. build on all branches, deploy only from main).
- Use conditions in Jenkinsfile.

POLLING vs WEBHOOKS

POLLING (Periodic Check)



- Jenkins checks the repository at regular intervals (e.g. every 1-5 minutes).
- Slower response time.
- Consumes resources.
- May delay builds.
- Useful when webhooks are not available.

WEBHOOKS (Event Driven)



- GitHub instantly notifies Jenkins when an event occurs (push, PR, etc.).
- Faster and more efficient.
- Builds start immediately.
- Recommended for production environments.

WHY USE WEBHOOKS?

- Real-time build triggers
- Lower load on Jenkins
- Faster feedback for developers
- Better for large teams and projects

INTEGRATION FLOW



Developer Push
Developer pushes code to a branch



GitHub
GitHub receives the push and triggers a webhook

Webhook



Jenkins
Jenkins receives the webhook event



Discover Branch
Jenkins discovers the branch (Multibranch)



Load Jenkinsfile
Jenkins loads the Jenkinsfile from the repo



Run Pipeline
Jenkins runs the pipeline for that branch

BEST PRACTICES

- Use webhooks instead of polling.
- Use Multibranch Pipelines.
- Test PR builds before merging.
- Use branch-based conditions.
- Keep your Jenkinsfile in the repository.
- Monitor webhook deliveries in GitHub.

COMMON WEBHOOK EVENTS

- Push (code is pushed to a branch)
- Pull request (opened, updated, closed)
- Branch created or deleted
- Tag created
- Repository updated



PRODUCTION TIP

- Use webhooks + Multibranch Pipelines for fast, scalable and reliable CI/CD.

TROUBLESHOOT MISSING WEBHOOK-TRIGGERED BUILDS

POSSIBLE CAUSE	HOW TO CHECK / FIX
Webhook not configured	Check GitHub repository → Settings → Webhooks
Incorrect Jenkins URL	Ensure the webhook URL is correct and accessible
Firewall or network issue	Verify Jenkins is reachable from GitHub (public URL or VPN)
Webhook events not selected	Select the correct events (e.g. Push, Pull request)
Jenkins not receiving events	Check Jenkins logs (Manage Jenkins → System Log)
Incorrect credentials	Verify webhook authentication if using a secret token
Branch filters blocking builds	Check branch or path filters in Multibranch configuration
Jenkins job not configured	Ensure the Multibranch Pipeline is properly set up
GitHub delivery failed	Check recent webhook deliveries in GitHub (Settings → Webhooks)

BUILDING AND TESTING APPLICATIONS

BUILD • TEST • VALIDATE • DELIVER • WITH CONFIDENCE

THE BUILD AND TEST PROCESS



EXAMPLE: MAVEN BUILD AND TEST

```
# Clean previous build
mvn clean

# Install dependencies
mvn install

# Run unit tests
mvn test

# Generate test report
# (Surefire runs automatically)

# Generate code coverage report
mvn jacoco:report
```

OTHER BUILD TOOLS



Maven

Popular for Java applications. Handles build, dependencies, testing and reports.



Gradle

Flexible build tool for Java, Kotlin and more.



Python (pip)

pip install -r requirements.txt
pytest



Node.js (npm)

npm install
npm test

SAMPLE TEST OUTPUT (MAVEN)

```
[INFO] Running Tests...
[INFO] Tests run: 15,
[INFO] Failures: 0,
[INFO] Errors: 0,
[INFO] Skipped: 0
[INFO] BUILD SUCCESS
```

CODE COVERAGE EXAMPLE

JaCoCo Report

Instructions : 85%
Branches : 78%
Lines : 83%
Methods : 87%

WHY FAILED TESTS SHOULD STOP DELIVERY



- A failed test usually means a bug in the code.
- Continuing the pipeline can deploy a broken application.
- Stops issues early and saves time and cost.
- Maintains trust in the delivery process.
- Only deploy code that meets defined quality standards.

BUILD EXIT CODES

Exit Code	Meaning
0	Success (Build and tests passed)
Non-zero (1, 2, etc.)	Failure (Build or tests failed)

COMMON BUILD AND TEST ISSUES



- Missing dependencies
- Compilation errors
- Test failures
- Low code coverage
- Incorrect Java/Python/Node version
- Build tool misconfiguration

BEST PRACTICES



- Keep build and test stages separate.
- Run tests in a clean environment.
- Publish test reports and coverage.
- Use consistent build tool versions.
- Fail fast on errors.
- Set quality gates (e.g. minimum coverage).



TROUBLESHOOTING TIP

- Jenkins often reports a failed pipeline because the underlying build tool returned a non-zero exit code.
- Check the build tool logs to find the real error.
- Look at the test report for failing tests.
- Common causes: failed tests, compilation errors, missing dependencies, or configuration issues.



KEY TAKEAWAY

A successful build is more than compiling code. It is proof that your application works, passes tests, and is ready for the next stage.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

LEARN
PRACTICE
GROW
WITH VERIQTA

ENGINEER TODAY. A BETTER TOMORROW.

ARTIFACTS AND ARTIFACT REPOSITORIES

BUILD ONCE • STORE SECURELY • DEPLOY EVERYWHERE

WHAT IS A BUILD ARTIFACT?



- A build artifact is the output of the build process.
- It can be a JAR, WAR, Docker image, ZIP file, binary, or any deployable file.
- It contains the application code and dependencies, ready to run in an environment.

WHY BUILD ONCE?



- Build once and promote the same artifact across environments (dev, staging, production).
- Ensures consistency, reliability and traceability.
- Avoids environment-specific bugs.
- Saves time and compute resources.

ARTIFACT VERSIONING



- Use meaningful versioning (e.g. 1.0.0, 1.2.3).
- Include build number, commit ID or timestamp.
- Helps with traceability and rollback.
- Follow semantic versioning where possible.

JENKINS ARTIFACTS vs REPOSITORIES



- Jenkins can archive artifacts for short-term use.
- Not designed for long-term storage or enterprise use.
- Use a dedicated artifact repository (e.g. JFrog Artifactory, Nexus) for production environments.

ARTIFACT REPOSITORY SYSTEMS



- JFrog Artifactory is a popular enterprise artifact repository.
- Supports multiple formats (Maven, Docker, npm, etc.).
- Provides access control, versioning, retention, and high availability.
- Alternatives: Sonatype Nexus, AWS CodeArtifact, GitHub Packages, Azure Artifacts.

IMMUTABLE ARTIFACTS



- Artifacts should be immutable (read-only).
- Once built, the artifact should never be changed.
- If changes are needed, create a new version.
- Prevents unexpected behavior and ensures integrity.

CHECKSUMS



- Generate checksums (e.g. SHA256) to verify artifact integrity.
- Ensures the artifact was not tampered with.
- Many repositories automatically generate and store checksums.

RETENTION



- Define retention policies (e.g. keep last 30 or 90 days).
- Clean up old artifacts to save storage space.
- Keep important releases (e.g. production) for longer periods.

COMMON ARTIFACT TYPES



- JAR** - Java applications
- WAR** - Web applications
- Docker Image** - Containerized apps
- ZIP** - Compressed files
- Binary** - Executables
- Other** - (npm, Python, etc.)

ARTIFACT FLOW



BEGINNER MISTAKE



- Rebuilding the application separately for production.
- Can lead to different artifacts, bugs and inconsistent behavior.
- Always promote the same artifact built earlier.

BEST PRACTICES



- Build once, promote many times.
- Use a dedicated artifact repository.
- Version your artifacts.
- Keep artifacts immutable.
- Use checksums.
- Set retention policies.
- Monitor storage usage.

PRODUCTION TIP



- Include the version number and commit ID in your artifact name.
- Store artifacts in a highly available repository.
- Restrict access to sensitive artifacts.
- Use artifact repositories for all environments, including production.

BUILDING CONTAINERS WITH JENKINS



AUTOMATE • BUILD • PACKAGE • DEPLOY • REPEAT

JENKINS + DOCKER WORKFLOW



- Jenkins automates the build, test and packaging of your application into a Docker image.
- It builds the image using a Dockerfile, tags it and pushes it to a container registry.
- The image is then deployed to your target environment (dev, staging, production).

WHAT IS A DOCKERFILE?



- A Dockerfile is a text file that contains instructions to build a Docker image.
- It defines the base image, application files, dependencies and how the application runs.
- It is versioned with your application code in Git.

BUILDING AN IMAGE



- Jenkins runs docker build to create the image.
- The image contains your application and all required dependencies.
- Each build can be tagged with a version, build number or commit SHA.

IMAGE TAGS



- Tags help identify different versions of the same image.
- Use meaningful tags such as:

- 1.0.0
- 2025.08.31
- `${GIT_COMMIT}` (commit SHA)
- `${BUILD_NUMBER}`

COMMIT SHA TAGGING



- Tagging images with the Git commit SHA ensures traceability.
- You can always trace which code produced the image.

Example:
`docker build -t myapp:${GIT_COMMIT} .`

CONTAINER REGISTRY



- A container registry stores your Docker images.
- Popular options:
 - Docker Hub
 - AWS ECR
 - Azure Container Registry
 - Google Artifact Registry
 - JFrog Artifactory
 - GitHub Container Registry (GHCR)

AUTHENTICATION



- Jenkins must authenticate to the registry before pushing images.
- Use Jenkins Credentials (Username/Password or Token).
- Do not hardcode credentials in the Jenkinsfile.

PUSHING IMAGES



- After a successful build, push the image to your registry.
- Example:

```
docker login -u $REGISTRY_USER \
-p $REGISTRY_TOKEN
docker push myapp:${GIT_COMMIT}
```

AVOID USING LATEST



- Do not use latest as the only tag for production.
- It can be overwritten, which makes deployments unpredictable.
- Always use specific tags (e.g. version, build number, commit SHA). You can still use latest for convenience but never as the only identifier.

JENKINS + DOCKER FLOW



Jenkins
(CI Pipeline)



Docker Build
(Build Image)



Image
(Tagged)



Registry
(Store Image)



Deployment
(Deploy to Environment)

SECURITY AWARENESS



- Never hardcode registry credentials in the Jenkinsfile.
- Use Jenkins Credentials and credential binding.
- Restrict access to your registry.
- Scan container images for vulnerabilities (e.g. Trivy, Snyk).

BEST PRACTICES



- Use meaningful and immutable tags.
- Tag with commit SHA and build number.
- Store images in a secure registry.
- Scan images before deployment.
- Keep Dockerfile in the application repo.
- Use least privilege for registry access.
- Automate the full flow in your pipeline.

TROUBLESHOOTING TIPS



- Check Docker daemon is available on the agent.
- Verify registry credentials.
- Ensure correct image tag and registry URL.
- Check network connectivity to the registry.
- Review Jenkins console logs.
- Confirm the image exists in the registry after push.

JENKINS CREDENTIALS AND SECRETS



KEEP YOUR SECRETS SAFE • BUILD SECURELY • FOLLOW BEST PRACTICES

JENKINS CREDENTIALS STORE



- Jenkins provides a secure credentials store to manage sensitive information.
- Credentials are stored encrypted in Jenkins.
- You can define credentials at different scopes (Global, Folder, or System).
- Use credentials instead of hardcoding secrets in pipeline scripts.

SUPPORTED CREDENTIAL TYPES



Secret text

API keys, tokens, passwords



Username/password

Usernames and passwords



SSH private keys

Private key for SSH access



Tokens

GitHub, AWS, GitLab, etc.



Others

Certificates, file credentials, etc.

CREDENTIAL IDS



- Each credential has a unique Credential ID.
- Use the Credential ID in your Jenkinsfile to reference the credential.
- Choose meaningful names (e.g. github-token, docker-creds).
- Keep IDs consistent across projects and environments.

Example ID:

github-token

withCredentials



- Use `withCredentials` to securely access credentials in a pipeline.
- Credentials are available only within the block.
- Values are automatically masked in the console output.

```
withCredentials([string(credentialsId: 'github-token',
variable: 'TOKEN')]) {
  sh 'echo "Using token"'
  sh 'curl -H "Authorization: token $TOKEN" \\\
https://api.github.com/user'
}
```

CREDENTIAL SCOPE



- **Global** – available to all jobs (on the controller).
- **Folder** – available only to jobs in a specific folder.
- **System** – used by Jenkins internally (plugins, agents).
- Use the smallest scope necessary for security (principle of least privilege).

MASKING



- Jenkins automatically masks credential values in console logs.
- Masked values appear as ****.
- Masking helps prevent accidental exposure in logs.
- However, masking does not make careless secret handling safe. Secrets can still be exposed in other ways (e.g. printing, files, debug output).

EXAMPLE: USING MULTIPLE CREDENTIALS



```
withCredentials([
  usernamePassword(credentialsId: 'docker-creds',
    usernameVariable: 'DOCKER_USER',
    passwordVariable: 'DOCKER_PASS'),
  string(credentialsId: 'api-token', variable: 'API_TOKEN')
]) {
  sh 'echo "Login to Docker"'
  sh 'echo $DOCKER_PASS | docker login -u $DOCKER_USER --password-stdin'
  sh 'curl -H "Authorization: Bearer $API_TOKEN" https://api.example.com'
}
```

EXTERNAL SECRET-MANAGEMENT SYSTEMS



- For enterprise environments, use external secret-management systems such as:
 - HashiCorp Vault
 - AWS Secrets Manager
 - Azure Key Vault
 - Google Secret Manager
 - CyberArk
 - JFrog Secrets
 - etc.
- Jenkins can integrate with these systems using plugins.

LEAST PRIVILEGE



- Give credentials only the minimum permissions required.
- Use read-only tokens where possible.
- Restrict access to specific repositories, registries, or environments.
- Regularly review and rotate credentials.
- Remove unused credentials.

COMMON FAILURE



- Credentials accidentally printed into build logs.
- Using `echo`, `set -x` or debug mode.
- Storing secrets in environment variables and then printing them.
- Hardcoding credentials in the Jenkinsfile.
- Using overly broad permissions.

TROUBLESHOOTING TIPS



- Check the console log for masked values (****).
- Ensure the correct Credential ID is used.
- Verify the credential has the right scope.
- Check plugin configuration if using external secret managers.
- Review your pipeline code for accidental exposure (e.g. `echo`, debug).
- Rotate credentials if they may have been exposed.

KEY TAKEAWAYS



- Use Jenkins credentials store.
- Never hardcode secrets.
- Use `withCredentials`.
- Follow least privilege.
- Consider external secret-management systems.
- Masking helps, but does not replace secure practices.

QUALITY GATES AND SECURITY GATES



BUILD SAFE • DEPLOY WITH CONFIDENCE • QUALITY BEYOND COMPILATION

WHY QUALITY AND SECURITY GATES?



- A successful build does not mean the application is ready for production.
- Quality and security gates help catch issues early.
- Prevent vulnerable, low-quality or non-compliant code from reaching production.
- Save time, cost and reputation.

KEY GATES EXPLAINED



BUILD

Compile the code and create build artifacts.



UNIT TESTS

Run automated unit tests to verify functionality.



CODE QUALITY

Analyze code with SonarQube to detect code smells, bugs and maintainability issues.



DEPENDENCY SCAN

Check open-source dependencies for known vulnerabilities (e.g. OWASP Dependency-Check, Snyk).



SECURITY SCAN

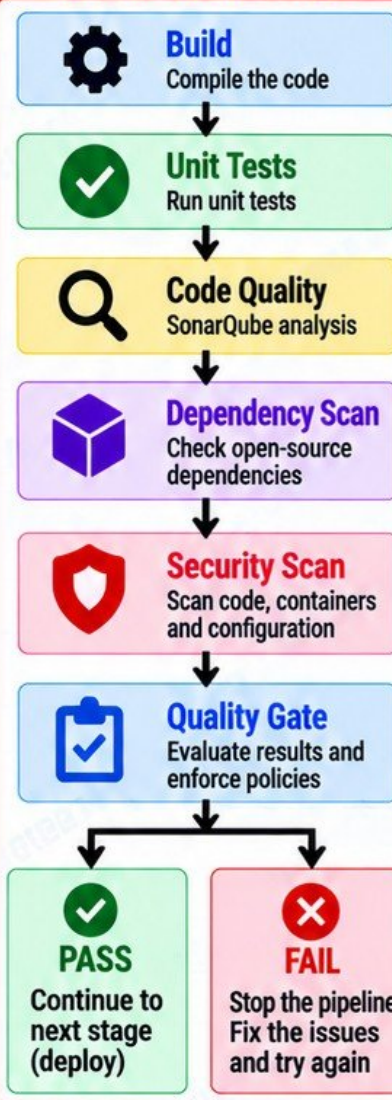
Scan application and container for security vulnerabilities (e.g. Trivy, Snyk, Clair).



QUALITY GATE

Evaluate results and enforce policies. Only proceed if all checks pass.

CI/CD PIPELINE WITH QUALITY AND SECURITY GATES



SONARQUBE INTEGRATION



SonarQube

- Analyzes code for bugs, vulnerabilities, code smells and maintainability.
- Configure quality gates (e.g. minimum coverage, no critical issues).
- Integrate with Jenkins using SonarQube Scanner plugin.
- Fail the pipeline if the quality gate is not met.

SOFTWARE COMPOSITION ANALYSIS



- Scans third-party libraries for known vulnerabilities.
- Tools: OWASP Dependency-Check, Snyk, Nexus IQ.
- Detects outdated or vulnerable dependencies.
- Generate reports and fail the build if high/critical vulnerabilities are found.

CONTAINER SCANNING



- Scan container images for vulnerabilities.
- Tools: Trivy, Snyk, Clair, Aqua.
- Integrate scanning into your pipeline before pushing to the registry.
- Block builds if critical vulnerabilities are detected.

POLICY GATES



- Enforce organizational policies (e.g. no critical vulnerabilities, minimum test coverage).
- Can include license compliance, security standards and infrastructure policies.
- Use tools like Open Policy Agent (OPA) or built-in quality gates.

PRODUCTION CONNECTION



- A successful build is not necessarily a releasable build.
- Only deploy what meets your quality, security and compliance standards.
- Protect your users, data and brand.

BEST PRACTICES



- Fail fast when a gate fails.
- Set measurable quality and security thresholds.
- Automate all checks in the pipeline.
- Keep quality and security tools updated.
- Monitor and review gate results regularly.
- Use separate gates for code, dependencies, containers and infrastructure.

SECURITY AWARENESS



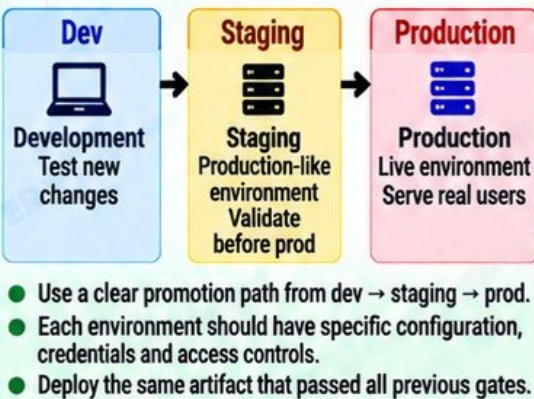
- Never skip security gates for convenience.
- Fix vulnerabilities instead of ignoring them.
- Do not treat warnings as low priority.
- Ensure security scanning happens before production deployment.
- A secure pipeline delivers a more secure application and protects your users and business.

DEPLOYING FROM JENKINS



AUTOMATE • CONTROL • DEPLOY • VERIFY • DELIVER VALUE

DEPLOYMENT STAGES



ENVIRONMENT-SPECIFIC CONFIGURATION

- Use environment-specific configuration (e.g. config files, Helm values, environment variables).
- Avoid hardcoding values in the pipeline.
- Use configuration management tools (e.g. Helm, Kustomize, Terraform, Ansible).
- Keep sensitive values in Jenkins credentials or external secret management systems.

DEPLOYMENT CREDENTIALS

- Use dedicated credentials for each environment.
- Store credentials in Jenkins Credentials Store.
- Use least privilege access.
- Examples: Kubernetes kubeconfig, cloud IAM credentials, SSH keys.
- Never hardcode credentials in the Jenkinsfile.

APPROVAL GATES

- Use manual or automated approval gates before deploying to production.
- Require approvals from authorized team members.
- Use Jenkins input step or external tools (e.g. Slack, GitHub, ServiceNow).
- Helps prevent accidental or unauthorized deployments.

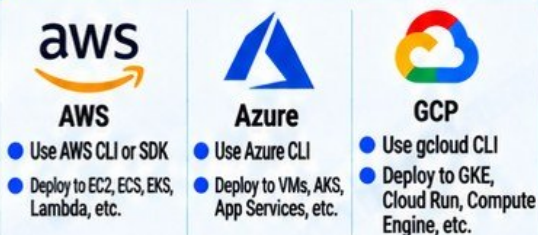
DEPLOYMENT VERIFICATION

- Verify the deployment after it completes.
- Check application health (health endpoint, logs, metrics).
- Run automated tests (smoke tests).
- Confirm the new version is running correctly.
- Monitor for errors and performance issues.

JENKINS AND KUBERNETES

- Use kubectl or Helm to deploy applications to Kubernetes.
- Manage kubeconfig as a Jenkins credential.
- Deploy using Kubernetes manifests or Helm charts.
- Verify rollout status (e.g. kubectl rollout status).
- Use namespaces for environment separation.

JENKINS AND CLOUD DEPLOYMENT TARGETS



WHY CONTROLLED AND REPEATABLE DEPLOYMENT MATTERS

- Ensures consistency across environments.
- Reduces human error.
- Makes deployments auditable.
- Allows quick rollback if needed.
- Supports disaster recovery.
- Enables teams to deploy with confidence and speed.
- Helps maintain system stability and reliability.

EXAMPLE: JENKINS DEPLOY STAGE (USING KUBERNETES)

```

stage('Deploy to Production') {
  steps {
    withCredentials([file(credentialsId: 'kubeconfig',
      variable: 'KUBECONFIG')]) {
      sh '''
        kubectl apply -f k8s/deployment.yaml
        kubectl rollout status deployment/myapp
      '''
    }
  }
}
  
```

PRODUCTION AWARENESS

- A successful deployment command does not prove the application is healthy.
- Always verify the application after deployment.
- Check logs, metrics and user functionality.
- Be ready to rollback if issues are detected.
- Monitor closely after deployment.

COMMON DEPLOYMENT ISSUES

- Incorrect configuration for environment.
- Missing or invalid credentials.
- Image not found in registry.
- Pod/container not starting.
- Service not accessible.
- Wrong namespace or resource name.
- Insufficient permissions.
- Deployment succeeds but application fails.

BEST PRACTICES

- Use the same build artifact across all environments.
- Keep deployment logic in version control.
- Use environment-specific configuration.
- Implement approval gates for production.
- Automate verification after deployment.
- Use monitoring and alerting.
- Document your deployment process.
- Keep it simple, repeatable and secure.

PRODUCTION APPROVALS AND DEPLOYMENT CONTROLS

PEOPLE • PROCESS • POLICIES • PROTECTION • PRODUCTION SAFETY

MANUAL APPROVAL WITH INPUT



- Use the input step in Jenkins to require manual approval before production deployment.
- You can also collect information such as change ticket number, reason for deployment, or approver name.

Example:

```
stage('Deploy to Production') {  
  steps {  
    input message: 'Deploy to production?',  
    parameters {  
      string(name: 'CHANGE_ID',  
        description: 'Enter change ticket ID')  
    }  
  }  
}
```

WHO SHOULD APPROVE?



- Production deployments should be approved by appropriate stakeholders.
- Examples: Release manager, team lead, operations team, or change advisory board (CAB).
- Avoid single-person approval for high-risk environments.
- Approvers should understand the impact and verify that all checks have passed.

SEPARATION OF DUTIES



- The person who develops code should not be the only person who deploys to production.
- Separate build, approval and deployment responsibilities where possible.
- Use different team roles and privileges in Jenkins.
- Enforce organizational policies and compliance requirements.
- This reduces the risk of accidental or malicious deployments.

PROTECTED PRODUCTION ENVIRONMENTS



- Restrict access to production jobs and environments.
- Use folder-based permissions, role-based access control (RBAC) and environment restrictions.
- Keep production credentials, API keys and cluster access tightly controlled.
- Use dedicated Jenkins agents with limited network access.

CHANGE CONTROLS



- Link deployments to a change ticket or work item.
- Follow your organization's change management process.
- Include details such as what is being deployed, who approved, when it will be deployed and how to roll back.
- Automate as much of the process as possible while keeping necessary approvals.

TIMEOUTS



- Set a timeout for manual approvals to avoid stuck pipelines.
- If approval is not received within the defined time, fail the pipeline.

Example:

```
stage('Approval') {  
  steps {  
    timeout(time: 30, unit: 'MINUTES') {  
      input message: 'Approve deployment'  
    }  
  }  
}
```

PREVENTING ACCIDENTAL DEPLOYMENTS



- Use explicit environment selection.
- Require manual approval for production.
- Use confirmation prompts.
- Restrict who can trigger production jobs.
- Validate that you are deploying the correct version and environment.
- Use tags, version checks and automated checks before deployment.

RESTRICTING PRODUCTION CREDENTIALS



- Store production credentials in Jenkins Credentials Store.
- Limit access to only authorized users and jobs.
- Use least privilege access.
- Use different credentials for each environment.
- Rotate credentials regularly.
- Do not hardcode credentials in the Jenkinsfile.

AUDITABILITY



- Keep a record of who approved, what was deployed, when it was deployed and which version.
- Integrate with your change management system.
- Use build logs, Jenkins audit logs and external tools (e.g. Slack notifications, ticket updates).
- This helps with compliance, incident investigation and accountability.

BEGINNER MISTAKE



- Allowing anyone who can trigger Jenkins to deploy directly to production.
- No approval process.
- No separation of duties.
- Using shared credentials.
- Skipping change management.
- This can lead to accidental deployments, outages and security incidents.

BEST PRACTICES



- Always use manual approval for production.
- Follow the principle of least privilege.
- Implement proper change controls.
- Use environment-specific credentials.
- Set timeouts for approvals.
- Keep an audit trail.
- Test the deployment process in non-production environments first.
- Make the deployment process repeatable and secure.

PRODUCTION AWARENESS

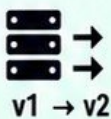


- Approval is not a delay. It is a protection mechanism.
- Production is a business-critical environment.
- A controlled deployment process reduces risk and improves reliability.
- Always verify after deployment, even if it was approved.

DEPLOYMENT STRATEGIES AND ROLLBACKS

DEPLOY SMART • MINIMIZE RISK • ENSURE AVAILABILITY • BE READY TO ROLLBACK

ROLLING DEPLOYMENT



v1 → v2

- Gradually replaces old instances with new ones.
- No downtime.
- Simple and widely used.
- Good for stateless applications.
- Risk: if the new version has issues, some users may experience problems during the rollout.

Use When:

- Small to medium changes
- Stateless applications
- You want zero downtime

BLUE-GREEN DEPLOYMENT



- Two identical environments (blue and green).
- Traffic is switched from blue to green when the new version is ready.
- Instant rollback by switching back if needed.
- Minimal downtime.

Use When:

- Critical applications
- You need fast and safe rollback
- You have sufficient infrastructure

CANARY DEPLOYMENT



- Release to a small percentage of users first (canary).
- Monitor metrics, logs and errors.
- Increase traffic gradually if healthy.
- Reduces risk by detecting issues early.
- Common in cloud and Kubernetes environments.

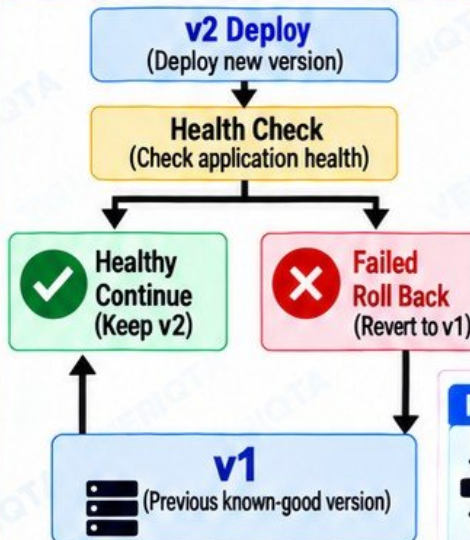
Use When:

- You want to test in production
- You need to minimize risk
- You have good monitoring in place

COMPARISON SUMMARY

Strategy	Downtime	Rollback Speed	Best Use Case
Rolling	None	Moderate	Regular releases, stateless apps
Blue-Green	Minimal	Very fast	Critical apps, zero downtime
Canary	None	Fast	Test with real users, minimize risk

DEPLOYMENT FLOW WITH ROLLBACK



KEY CONCEPTS

- Health verification ensures the application is working after deployment.
- Use metrics, logs and synthetic checks.
- Automate rollback triggers where possible.
- Always keep a previous known-good version.
- Rollback must be designed and tested before an incident occurs, not during the incident.

FAILED DEPLOYMENT DETECTION



- Application health checks fail (HTTP 5xx, failed readiness).
- Error rate increases.
- Latency or response time degrades.
- Key business metrics drop.
- Pods or instances crash or restart.
- Automated monitoring alerts.

WHY ROLBACK PLANS MATTER



- Reduces downtime and business impact.
- Helps you respond quickly to failures.
- Prevents panic and rushed decisions.
- Must be planned, automated and tested regularly.
- A successful deployment command does not mean the application is healthy.

BEST PRACTICES



- Choose the right strategy for your use case.
- Automate health checks and rollback.
- Keep deployment process repeatable.
- Use infrastructure as code (e.g. Helm, Terraform).
- Test rollback regularly.
- Monitor application and infrastructure metrics.
- Document your deployment and rollback process.

ROLLBACK TRIGGERS



- Health check failure.
- Error rate above threshold.
- High latency or failed transactions.
- Manual trigger by operator.
- Automated trigger from monitoring tools (e.g. Prometheus, CloudWatch).

PRODUCTION AWARENESS



- A deployment can succeed but the application can still be broken.
- Always verify after deployment.
- Have clear rollback procedures.
- Monitor closely after each release.
- Communicate with stakeholders.
- Be ready to roll back quickly if needed.

HANDLING FAILURES IN JENKINS PIPELINES

DETECT • INVESTIGATE • RECOVER • IMPROVE



STAGE FAILURES



- A stage can fail due to a command error, test failure, deployment issue, or timeout.
- When a stage fails, the pipeline moves to the post actions.
- Identify which stage failed and why using the Jenkins logs.
- Use clear stage names for easy troubleshooting.

EXIT CODES



- Commands return an exit code to indicate success or failure.
- 0 = success
- Non-zero (e.g. 1, 2, 126, 127) = failure
- Jenkins marks the stage as FAILED when a command returns a non-zero exit code.
- Example: a failed unit test or build tool will return a non-zero exit code.

post { } SECTION



- The post block defines actions to run based on the pipeline result.
- Common conditions:
 - **always** - runs no matter what
 - **success** - runs only if the pipeline succeeds
 - **failure** - runs only if the pipeline fails
 - **unstable** - runs if the pipeline is unstable (e.g. test failures)
- Use post to send notifications, archive logs, clean up resources, or perform other tasks.

TIMEOUTS



- Use timeouts to prevent pipelines from running forever.
- Set stage-level or pipeline-level timeouts.
- Helps free up build agents and resources.

Example:

```
stage('Build') {
  options { timeout(time: 10, unit: 'MINUTES') }
  steps {
    sh 'mvn clean install'
  }
}
```

RETRIES



- Use retries for temporary failures (e.g. network issues, flaky tests).
- Do not blindly retry all failures.
- Set a reasonable number of retries.

Example:

```
stage('Test') {
  steps {
    retry(3) {
      sh 'mvn test'
    }
  }
}
```

Example:

```
pipeline {
  ...
  post {
    always { echo 'Always run' }
    success { echo 'Build succeeded' }
    failure { echo 'Build failed' }
    unstable { echo 'Build unstable' }
  }
}
```

CLEANUP



- Always clean up after a failure or successful run.
- Remove temporary files, workspaces, containers, and cloud resources.
- Use post { always { ... } } for cleanup tasks.

Example:

```
post {
  always {
    cleanWs() // clean workspace
  }
}
```

WHY BLINDLY RETRYING FAILURES CAN HIDE REAL PROBLEMS



- Some failures are not temporary (e.g. code errors, misconfigurations, failing tests).
- Blind retries can waste time, compute resources and hide the real issue.
- Only retry known temporary failures (e.g. network or external service issues).
- Always investigate the root cause of repeated failures.

COMMON FAILURE CAUSES



- Build tool errors
- Unit test failures
- Dependency download issues
- Environment or configuration problems
- Deployment failures
- Insufficient permissions
- External service outages
- Timeouts

BEST PRACTICES



- Set appropriate timeouts and retries.
- Use post actions for cleanup and notifications.
- Collect and keep useful logs and artifacts.
- Investigate and fix the root cause.
- Do not ignore unstable builds.
- Use clear stage names and meaningful logs.
- Monitor pipeline health and failure trends.
- Automate notifications (Slack, email, etc.).

FAILURE FLOW

1

Failure

Pipeline or stage fails

2

Capture Evidence

Logs, artifacts, error messages

3

Identify Failed Stage

Check Jenkins console output

4

Inspect Logs

Look for error details

5

Diagnose

Find the root cause

6

Fix

Update code, configuration or environment

7

Re-run

Trigger the pipeline again

8

Verify

Ensure the pipeline and application work



LEARN
PRACTICE
GROW
WITH VERIQTA

TROUBLESHOOTING JENKINS IN PRODUCTION

FIND PROBLEMS • ANALYZE CAUSES • APPLY FIXES • KEEP PIPELINES RUNNING



COMMON REAL SCENARIOS

- 1 Pipeline stuck in queue**
Jobs stay in queue and never start.
- 2 Agent offline**
Build agent appears offline or disconnected.
- 3 Workspace permission denied**
Permission errors when accessing workspace.
- 4 Git checkout failure**
Cannot clone or fetch code from repository.
- 5 Credentials rejected**
Authentication failure for Git, registry or other services.
- 6 Dependency download failure**
Maven/Gradle/npm dependencies cannot be downloaded.
- 7 Docker build failure**
Error building Docker image in pipeline.
- 8 Registry push denied**
Cannot push image to container registry.
- 9 Quality gate failure**
Pipeline fails due to SonarQube or other quality gate.
- 10 Deployment timeout**
Deployment takes too long and times out.
- 11 Disk space exhausted**
Jenkins master or agent out of disk space.
- 12 Plugin-related failure**
Job fails due to plugin errors or incompatibility.

EXAMPLE: AGENT OFFLINE



- **Symptom:** Agent shows offline.
- **Evidence:** Agent log shows disconnection.
- **Hypothesis:** Network issue or agent service stopped.
- **Diagnostic Check:** Check agent logs and network connectivity.
- **Cause:** Agent service not running.
- **Fix:** Start the agent service and verify connectivity.
- **Verification:** Agent comes online and builds run successfully.

KEY TAKEAWAY

Troubleshooting Jenkins is about a systematic and structured approach. Find the real cause, apply the right fix, and prevent it from happening again.

TROUBLESHOOTING METHODOLOGY

- 1 Symptom**
What is the issue? (Be specific)
- 2 Evidence**
Check logs, UI, error messages and system state.
- 3 Hypothesis**
What could be the possible cause?
- 4 Diagnostic Check**
Run commands or checks to confirm the cause.
- 5 Cause**
Identify the root cause.
- 6 Fix**
Apply the correct fix.
- 7 Verification**
Confirm the issue is resolved and the pipeline runs successfully.

EXAMPLE: GIT CHECKOUT FAILURE



- **Symptom:** Git checkout fails.
- **Evidence:** Authentication error in logs.
- **Hypothesis:** Invalid credentials or missing SSH key.
- **Diagnostic Check:** Test Git access manually.
- **Cause:** Expired token or wrong key.
- **Fix:** Update credentials in Jenkins.
- **Verification:** Pipeline clones the repository successfully.

BEST PRACTICES



- Monitor Jenkins and agents.
- Set up log rotation and cleanup.
- Keep plugins updated and tested.
- Use infrastructure as code.
- Implement alerts and notifications.
- Maintain a troubleshooting runbook.

USEFUL LOG LOCATIONS



- Pipeline console output
- Jenkins system logs (/var/log/jenkins)
- Agent logs
- Build tool logs (Maven, Gradle, etc.)
- Docker build logs
- Application deployment logs
- Plugin logs
- Server and OS logs (journalctl, syslog)

USEFUL COMMANDS (EXAMPLES)



- Check Jenkins service
`systemctl status jenkins`
- Check disk space
`df -h`
- Check running processes
`ps aux | grep jenkins`
- Check agent connectivity
`ping <agent-ip>`
- Check Docker
`docker info`
- Check Git access
`ssh -T git@github.com`
- Check network connectivity
`curl -I https://repo.maven.org`

QUICK TROUBLESHOOTING TIPS



- Always check the console logs first.
- Look for the first error, not the last.
- Verify credentials and permissions.
- Check disk space and system resources.
- Ensure plugins are up to date.
- Test commands manually on the agent.
- Use small, incremental changes.
- Keep pipelines simple and modular.
- Document common issues and fixes.

PRODUCTION AWARENESS



- Jenkins is a critical production service.
- Monitor Jenkins health and resource usage.
- Set up alerts for failures and downtime.
- Keep backups of Jenkins configuration.
- Regularly clean old builds and workspace.
- Have a runbook for common issues.
- Practice troubleshooting before an incident.
- Minimize single points of failure.

MONITORING AND OPERATING JENKINS

OBSERVE • IDENTIFY • ALERT • MAINTAIN • KEEP IT RELIABLE



JENKINS HEALTH



- Check overall Jenkins health from the dashboard.
- Ensure the controller and agents are online and healthy.
- Monitor for warnings and critical errors.
- Keep Jenkins updated and secure.

CONTROLLER RESOURCES



- Monitor CPU usage.
- Monitor memory (RAM) usage.
- Monitor disk usage.
- Monitor JVM (Java) performance.
- Ensure sufficient resources for plugins, builds and concurrent jobs.

AGENT HEALTH



- Check agent status (online/offline).
- Monitor resource usage on agents (CPU, memory, disk).
- Ensure agents can connect to the controller.
- Investigate disconnected agents quickly.
- Use labels and monitoring to track agent availability.

KEY METRICS TO MONITOR



- Queue length (number of jobs waiting).
- Build duration (trends and outliers).
- Failure rate (failed vs successful builds).
- Executor utilization (how busy agents are).
- Disk usage (controller and agents).
- Build retention (storage consumption).

LOGS



- Jenkins system logs (\$JENKINS_HOME/logs).
- Controller logs.
- Agent logs.
- Build logs.
- Look for errors, stack traces and repeated warnings.
- Centralize logs for easier troubleshooting.

METRICS, PROMETHEUS AND GRAFANA



- Jenkins → Prometheus → Grafana
- Use Prometheus plugin to expose Jenkins metrics.
 - Collect metrics like build count, duration, queue length, executor usage, JVM metrics, etc.
 - Visualize metrics in Grafana dashboards.
 - Set up alerts for failures, high queue length, resource usage and offline agents.

ALERTING



- Set up alerts for critical issues (e.g. high failure rate, controller down, agents offline).
- Use email, Slack or other notification channels.
- Alert on disk space, high queue length, and long build durations.
- Respond to alerts quickly to prevent bigger problems.

BUILD RETENTION



- Keep builds only for a defined period (e.g. 30 days).
- Use build discarder policies.
- Delete old artifacts and logs automatically.
- Balance between storage cost and the need for historical data.

BACKUP STRATEGY



- Regularly back up:
 - Jenkins configuration
 - Jobs and pipelines
 - Plugins
 - Credentials (securely)
 - Build history (if needed)
- Store backups off the Jenkins server.
- Test your restore process regularly.
- A backup is critical for disaster recovery.

WHY JENKINS IS PRODUCTION INFRASTRUCTURE



- When your application deployments depend on Jenkins,
- Jenkins itself becomes production infrastructure.
- A Jenkins outage can stop software delivery, causing business impact.
- It must be monitored, secured, backed up and operated like any other critical production system.

BEST PRACTICES



- Monitor Jenkins and agents continuously.
- Use metrics, dashboards and alerting.
- Keep Jenkins updated and secure.
- Manage disk space and build retention.
- Have a tested backup and recovery plan.
- Document operational procedures.
- Treat Jenkins as a critical production service.

SECURING AND HARDENING PRODUCTION JENKINS



BUILD A SECURE JENKINS • PROTECT CREDENTIALS • REDUCE RISK • KEEP IT RELIABLE

AUTHENTICATION



- Use strong authentication methods (LDAP, SAML, OAuth).
- Enforce multi-factor authentication (MFA).
- Disable or remove default accounts.
- Use strong password policies.
- Integrate with your organizational identity provider (e.g. Active Directory).

AUTHORIZATION



- Implement role-based access control (RBAC).
- Follow the principle of least privilege.
- Give users only the permissions they need.
- Separate admin, developer and deployment roles.
- Regularly review and remove unused access.

CREDENTIAL PROTECTION



- Store credentials in Jenkins Credentials Store.
- Use credential bindings (e.g. with Credentials).
- Never hardcode secrets in Jenkinsfiles.
- Restrict access to sensitive credentials.
- Rotate secrets regularly.
- Monitor for unusual credential usage.

PLUGIN MANAGEMENT



- Install only trusted and necessary plugins.
- Keep plugins updated.
- Remove unused or outdated plugins.
- Review plugin permissions.
- Avoid installing random plugins from untrusted sources.

PATCH MANAGEMENT



- Keep Jenkins core updated regularly.
- Apply security patches quickly.
- Monitor security advisories for Jenkins and plugins.
- Test updates in a non-production environment.
- Plan a regular update schedule.

CONTROLLER ISOLATION



- Run the controller on a dedicated and secured server.
- Limit direct internet access.
- Use a firewall and network segmentation.
- Restrict SSH access.
- Monitor system resources and logs.

AGENT TRUST BOUNDARIES



- Use trusted and isolated agents.
- Do not allow untrusted code to run on the controller.
- Use separate agents for untrusted builds.
- Restrict agent permissions and network access.
- Consider ephemeral agents (e.g. in Kubernetes).

NETWORK RESTRICTIONS



- Allow access only from trusted networks (VPN, office, CI/CD).
- Use firewall rules to restrict access.
- Do not expose Jenkins publicly unless absolutely necessary.
- Restrict inbound and outbound traffic.
- Use a reverse proxy if external access is required.

HTTPS AND CSRF PROTECTION



- Always use HTTPS with a valid SSL/TLS certificate.
- Redirect HTTP to HTTPS.
- Keep CSRF protection enabled (don't disable it).
- Use a trusted certificate authority or internal PKI.
- Regularly renew certificates.

SCRIPT APPROVALS



- Review and approve scripts carefully.
- Only approve trusted code.
- Understand what each script does before approving.
- Use the in-process script approval system.
- Avoid approving unnecessary permissions.

AUDIT LOGGING



- Enable audit logging.
- Track user activity, configuration changes and credential usage.
- Integrate with centralized logging (e.g. ELK, Splunk).
- Regularly review logs for suspicious activity.
- Set up alerts for critical changes (e.g. new admin user).

BACKUP PROTECTION



- Regularly back up Jenkins configuration, jobs, plugins and credentials (encrypted).
- Store backups in a secure location (with access control).
- Test restore procedures regularly.
- Protect backups from unauthorized access.
- Include Jenkins home, plugins and job configurations.

SECRET ROTATION



- Rotate credentials, API keys, tokens and certificates regularly.
- Automate rotation where possible.
- Update dependent systems after rotation.
- Remove old and unused secrets.

AVOIDING PUBLIC EXPOSURE



- Do not expose Jenkins directly to the internet.
- Use VPN, private networks or a bastion host.
- Implement strict firewall rules.
- Use IP allowlists if external access is required.
- Regularly scan for open ports and vulnerabilities.

PRODUCTION AWARENESS



- Jenkins can hold powerful credentials (e.g. cloud, servers, production systems).
- Compromising Jenkins can become an infrastructure compromise.
- Treat Jenkins as a critical production asset.
- Secure, monitor and maintain it like any other production system.

COMPLETE PRODUCTION JENKINS PIPELINE PROJECT

FROM CODE TO PRODUCTION • SECURE • AUTOMATED • REAL-WORLD

SAME PIPELINE
REAL IMPACT

PROJECT OVERVIEW



- Build a real-world CI/CD pipeline using Jenkins from code commit to production deployment.
- Integrate testing, security, quality gates and container deployment.
- Use best practices for security, monitoring and rollback.
- This project simulates a real enterprise pipeline.

TOOLS AND TECHNOLOGIES



GitHub (Source Code)



Jenkins (CI/CD)



Maven (Build)



JUnit (Unit Tests)



JaCoCo (Code Coverage)



SonarQube (Code Quality)



Trivy / OWASP (Security Scan)



Docker (Container Image)



Container Registry (e.g. Docker Hub, ECR)



Kubernetes / Cloud (Deployment)



Grafana (Monitoring)

KEY LEARNING OUTCOMES



- Set up a production-ready Jenkins pipeline.
- Integrate multiple tools and quality gates.
- Build and deploy containerized applications.
- Implement security and approval controls.
- Monitor the deployed application.
- Handle failures and rollbacks.
- Apply real-world DevOps best practices.

PRODUCTION MINDSET

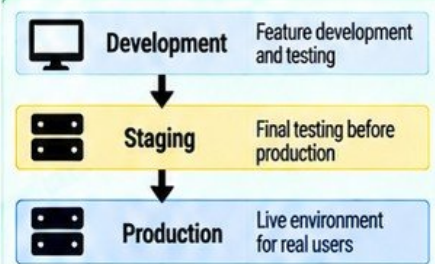


- A successful build is not a successful deployment.
- Security and quality gates are mandatory.
- Automate everything that can be automated.
- Keep deployments controlled and repeatable.
- Always have a rollback plan.
- Monitor after deployment.
- Treat your pipeline as production infrastructure.

END-TO-END PIPELINE FLOW



ENVIRONMENT FLOW



IMPORTANT CONFIGURATIONS



- Webhook integration with GitHub
- Jenkins credentials and secrets
- Environment-specific configuration
- Tool installations on agents
- Quality gates (e.g. SonarQube)
- Container registry credentials
- Deployment scripts (e.g. Helm, kubectl)
- Approval gates for production

VERIFICATION AND MONITORING



- Check application health endpoints
- Verify logs and metrics (Prometheus)
- Visualize dashboards (Grafana)
- Set up alerts for failures
- Confirm application functionality
- Monitor performance and errors

ROLLBACK STRATEGY



- Keep previous known-good version
- Automate rollback process
- Use version tags and immutable images
- Test rollback procedure regularly
- Have a manual rollback option

FINAL RESULT



- A secure, automated, production-ready CI/CD pipeline.
- Code is tested, scanned, and approved before deployment.
- Application is deployed, monitored, and ready to scale.
- You have hands-on experience with a real DevOps pipeline.
- You are ready for real-world production environments.

BUILD
DEPLOY
LEARN
REPEAT



ENGINEER TODAY. A BETTER TOMORROW.

LEARN
PRACTICE
GROW
WITH VERIQTA